

FastAPI School Management System

Project Overview

This is a comprehensive full-stack web application built during an internship at **Nexsofture** for managing school data including students, classes, and subjects. The project demonstrates professional backend development practices using **FastAPI**, **SQLModel**, and **SQLite**, combined with frontend templates using **Jinja2** for server-side rendering.

Project Name: Internship-Nex-Fastapi

Technology Stack:

- Backend Framework: FastAPI (Python)
- ORM: SQLModel (Pydantic + SQLAlchemy)
- Database: SQLite
- Templating Engine: Jinja2
- Server: Uvicorn

Architecture Overview

The project follows a well-organized **layered architecture** pattern with clear separation of concerns:

FastAPI School Management System

- ├── Controllers (API Endpoints)
- ├── Jinja Controllers (Template Responses)
- ├── Models (Pydantic Request/Response Schemas)
- ├── DAL (Data Access Layer)
- ├── Schema (SQLModel Database Models)
- ├── Configuration (Dependencies & Database)
- └── Templates (HTML Frontend)

Database Schema

Core Tables

The system manages three main entities with UUID primary keys and automatic timestamps:

1. Student Table

Stores comprehensive student information with blood group tracking and class assignment.

Fields:

- id (UUID): Primary key with auto-generated UUID
- name (String): Student's full name

- email (String): Unique email address
- primary_contact_number (String): Contact phone number
- blood_group (Enum): Blood group (B+, B-, A+, AB+, O+, O-)
- class_std (Integer): Class standard (1-12)
- date_of_birth (Date): Student's date of birth
- created_at (DateTime): Auto-generated creation timestamp
- updated_at (DateTime): Auto-updated modification timestamp

2. Class Table

Manages class information and relationships with subjects.

Fields:

- id (UUID): Primary key with auto-generated UUID
- class_name (Integer): Class standard (1-12)
- class_teacher (String, Optional): Name of class teacher
- created_at (DateTime): Auto-generated creation timestamp
- subjects (Relationship): One-to-many relationship with Subject table

3. Subject Table

Tracks subjects assigned to classes.

Fields:

- id (UUID): Primary key with auto-generated UUID
- subject_name (String): Name of the subject
- class_id (UUID, Foreign Key): Reference to Class table
- created_at (DateTime): Auto-generated creation timestamp
- classes (Relationship): Back-reference to Class table

Database Configuration:

- Engine: SQLite (file-based: creator.db)
- Echo Mode: Enabled (logs SQL queries for debugging)
- Auto-migration: Tables created on application startup

Layer-Wise Architecture

1. Schema Layer ([schema.py](#))

Defines SQLAlchemy database models with complete CRUD capabilities. Includes:

- **UUID Base Class:** Provides default UUID primary key
- **BloodGroup Enum:** Defines valid blood group values
- **Student, Class, Subject Models:** Complete SQLAlchemy ORM mapping with validation constraints

2. Models Layer ([StudentModels.py](#), [ClassModels.py](#), [SubjectsModels.py](#))

Pydantic models for request/response validation:

Student Models:

- `StudentRequest`: Input schema for creating students
- `StudentResponse`: Output schema with timestamps
- `StudentUpdate`: Partial update schema
- `StudentContact`: Contact information update
- `StudentClass`: Class assignment update
- `StudentForm`: HTML form data schema

Class Models:

- `ClassRequest`: Input schema for creating classes
- `ClassResponse`: Output schema with relationships

Subject Models:

- `SubjectRequest`: Input schema for creating subjects
- `SubjectResponse`: Output schema
- `SubjectClass`: Join model for Student-Class relationships

3. Data Access Layer (DAL)

[GenericDal.py](#) - Generic base class with CRUD operations:

Operations:

- `create(obj)`: Insert new record and return created object
- `get(id)`: Fetch single record by UUID
- `get_all()`: Fetch all records as list
- `update(id, obj_data)`: Update specific fields (null values ignored)
- `delete(id)`: Remove record and return success boolean

Specialized DAL Classes:

- `StudentDal`: Extends `GenericDal` with custom `count_class()` method for grouping students by class
- `ClassDal`: Extends `GenericDal` for class-specific operations
- `SubjectDal`: Extends `GenericDal` with `Jointable()` method for joined Subject-Class queries

4. Controller Layer

API Controllers (JSON Responses)

[StudentController.py](#)

- `POST /Student/` - Create new student
- `GET /Student details` - Get all students
- `PUT /Student_update/{student_id}` - Update student details
- `PUT /{student_id}` - Update student contact
- `PUT /increment/{student_id}` - Increment student class

- PUT /Decrement/{student_id} - Decrement student class
- GET /class_student - Count students per class

ClassController.py

- POST /Class - Create new class
- GET /classes/{class_id} - Get class by ID
- GET /class_details - Get all classes
- PUT /Class_update/{class_id} - Update class details
- DELETE /delete/{class_id} - Delete class

SubjectController.py

- POST /Subject/ - Create new subject
- GET /subject_result/{subject_id} - Get subject by ID
- GET /Subject_details - Get all subjects
- PUT /Subject_update/{subject_id} - Update subject
- DELETE /Sub_delete/{subject_id} - Delete subject
- GET /Relation_table - Get joined Subject-Class data

Jinja2 Controllers (HTML Template Responses)

StudentJinja.py - Server-side rendered pages for student management:

Navigation Routes:

- GET / - Home page
- GET /Student - Student home page
- GET /About - About page

Student Form Routes:

- GET /Studentform - Student registration form page
- POST /submit - Process student registration
- GET /Studentupdate - Update student page
- POST /updatesubmit - Process student update
- GET /Studentclass - Class assignment page
- POST /updateclass - Update class assignment
- GET /Studentcontact - Contact update page
- POST /updatecontact - Update contact information
- GET /Studentdetails - Display all students

ClassJinja.py - Server-side rendered pages for class management:

- GET /Class - Class home page
- GET /Classcreate - Class creation form
- POST /createclass - Create class
- GET /Classgetid - Find class form
- POST /classfind - Find specific class
- GET /ClassUpdate - Update class page
- POST /classupdate - Update class details
- GET /Classdelete - Delete class form
- POST /classdelete - Delete class
- GET /ClassDetails - List all classes

SubjectJinja.py - Server-side rendered pages for subject management:

- GET /Subject - Subject home page
- GET /Subjectcreate - Subject creation form
- POST /createform - Create subject
- GET /Subjectfind - Find subject form
- POST /subjectfind - Find specific subject
- GET /SubjectUpdate - Update subject page
- POST /updatesubject - Update subject details
- GET /Subjectdeleteform - Delete subject form
- POST /subjectdelete - Delete subject
- GET /SubjectDetails - List all subjects

5. Configuration Layer

DatabaseConfig.py

Handles database connectivity and session management:

- create_engine(): SQLite connection setup with echo=True
- create_tables(): Auto-creates all SQLAlchemy tables on startup
- get_db(): Context manager for database sessions (dependency injection)

Dependency Injection Files:

- StudentDataDependencyConfig.py: Provides get_student_dal() dependency
- ClassDataDependencyConfig.py: Provides get_class_dal() dependency
- SubjectDataDependencyConfig.py: Provides get_subject_dal() dependency

These enable automatic dependency injection in route handlers via FastAPI's Depends() mechanism.

6. Common Utilities

CommonModel.py

Generic response wrapper for all API endpoints:

ResponseTo[T]:

- message: str (success/failure description)
- status: str | int (HTTP status code)
- response: Optional[T] (actual data payload)

Provides consistent JSON response format across all endpoints.

Application Entry Point (**main.py**)

Server Configuration:

- Host: 127.0.0.1 (localhost)
- Port: 8000
- Auto-reload: Enabled for development

Startup Event:

```
@app.on_event("startup")
def on_startup():
    create_tables() # Auto-creates all database tables
```

Static File Mounts:

- /static - Student, Class, Subject template assets
- /ClassPhotos - Class-related images
- /Photos - Student profile pictures
- /SubPhotos - Subject-related media

Route Registration:

```
app.include_router(apirouter) # Student Jinja routes
app.include_router(classter) # Class Jinja routes
app.include_router(suberout) # Subject Jinja routes
app.include_router(router) # Student API routes
app.include_router(classrouter) # Class API routes
app.include_router(subrouter) # Subject API routes
```

Key Features and Functionality

1. Student Management

Create: Register new students with complete information including blood group and date of birth

Read:

- View all students
- Get student count grouped by class
- Fetch individual student details

Update:

- Modify student personal information (name, DOB, blood group)
- Update class assignment
- Update contact information

Delete: Remove student records (implicit in controllers)

Special Operations:

- Class increment/decrement for student progression
- Blood group tracking for medical records
- Unique email constraint for student identification

2. Class Management

Create: Define new classes (1-12 standard) with assigned teacher

Read:

- View all classes

- Fetch individual class with relationships

Update: Modify class name and teacher assignment

Delete: Remove class records

Relationships: Bidirectional relationship with subjects

3. Subject Management

Create: Add new subjects with class assignment

Read:

- View all subjects
- Fetch individual subject
- View joined Subject-Class data

Update: Modify subject name and class assignment

Delete: Remove subject records

Relationships: Belongs to specific Class

4. Template-Based Frontend

Uses Jinja2 templating for:

- Server-side HTML rendering
- Form submissions with POST requests
- Error handling and validation responses
- Dynamic content rendering
- Static asset serving (CSS, images)

Design Patterns Used

1. Generic DAL Pattern

Eliminates code duplication with parameterized CRUD operations applicable to all entities.

2. Dependency Injection

FastAPI's Depends() automatically provides database sessions and DAL instances to route handlers.

3. Request/Response Models

Pydantic models ensure type safety and automatic validation on input/output.

4. Layered Architecture

Clear separation: Controllers → Models → DAL → Database Schema

5. UUID Primary Keys

Globally unique identifiers for better security and distribution.

6. Relationship Mapping

SQLModel relationships enable:

- Automatic eager/lazy loading
- Cascade operations
- Referential integrity

Request/Response Flow Example

Creating a Student:

User submits HTML form

↓

StudentJinja.post(/submit)

↓

Dependency Injection: get_student_dal()

↓

StudentDal.create(Student object)

↓

GenericDal.create() - Database INSERT

↓

Return StudentResponse with created_at/updated_at

↓

Render submitform.html template with response

Database Constraints and Validation

Student Validations:

- Email: Unique constraint
- Blood Group: Enum validation (6 valid values)
- Class Standard: Integer between 1-12
- All fields: Type validation via Pydantic

Class Validations:

- Class Name: Integer 1-12
- Optional class teacher field

Subject Validations:

- Subject Name: Required string
- Optional class assignment (foreign key)

Auto-Fields:

- UUID: Auto-generated on creation
- created_at: Set on insert
- updated_at: Set on insert, updated on modification

Error Handling

Standard HTTP Status Codes:

- 201 Created: Successful resource creation
- 200 OK: Successful retrieval/update/deletion
- 404 Not Found: Resource not found for update/delete
- Form errors: Redirects to error template with message

Generic Error Responses:

```
ResponseTo(  
message="Not Found",  
status=404,  
response=None  
)
```

Future Enhancement Opportunities

1. **Authentication & Authorization:** Implement JWT-based user authentication with role-based access control
2. **Advanced Validations:** Email verification, phone number validation, age restrictions
3. **Search & Filtering:** Implement advanced search by class, blood group, date range
4. **Pagination:** Add pagination for large datasets
5. **File Uploads:** Student photo uploads with validation
6. **Notifications:** Email/SMS notifications for parent-school communication
7. **Analytics Dashboard:** Student enrollment trends, class distribution analytics
8. **Soft Deletes:** Implement soft delete for data recovery
9. **Audit Logging:** Track all modifications with timestamps and user information
10. **API Rate Limiting:** Implement rate limiting for API endpoints
11. **Integration Tests:** Comprehensive test coverage with pytest
12. **Docker Containerization:** Production-ready deployment configuration

Project Development Insights

This project demonstrates:

- ✓ Professional backend API development with FastAPI
- ✓ Proper database modeling with SQLAlchemy
- ✓ Clean code architecture with layered design
- ✓ Reusable generic components (GenericDal)
- ✓ Server-side rendering with Jinja2
- ✓ Dependency injection pattern
- ✓ RESTful API design principles
- ✓ Both JSON APIs and HTML template responses

- ✓ Comprehensive CRUD operations
- ✓ Relationship mapping and joins
- ✓ Timestamp tracking for auditing

Conclusion

This FastAPI School Management System serves as an excellent portfolio project demonstrating full-stack web development capabilities. The clean architecture, separation of concerns, and use of modern Python frameworks showcase professional-grade software engineering practices. The dual approach of providing both RESTful APIs and server-rendered HTML templates makes it adaptable for various use cases from mobile-friendly SPAs to traditional web applications.